Part 1: Text Processing and Exploratory Data Analysis

Group members: Júlia Hernan (u233681), Alan Le Roux (u233515), Alba Miró (u233727)

GitHub repository: https://github.com/juliahupf/RIAW.git

1. Introduction

This report describes the first phase of our RAG system built on a custom search engine: the preparation of the e-commerce fashion products corpus and its exploratory analysis. The corpus contains 28080 product documents. Each document has the fields pid, title, description, brand, category, sub_category, product_details, seller, out_of_stock, selling_price, discount, actual_price, average_rating, url, plus images and crawl metadata.

The work is implemented in Python and lives in the repository above. Section 5 explains how to reproduce every result.

2. Data preparation

2.1 Text pre-processing (Task 1)

We pre-process the text fields title and description with one shared function, so that documents and (in later phases) queries go through exactly the same analysis. The steps, in order, are:

1. **Case folding.** All text is lowercase so that "Cotton" and "cotton" match.
2. **URL removal.** URLs add no meaning to a product description.
3. **Punctuation removal.** Punctuation is replaced by a space instead of being deleted. We found descriptions with a missing space after a period (for example "look.Designed"), and deleting the period would have fused two words into one token.
4. **Tokenization.** After cleaning, tokens are obtained by splitting on whitespace.
5. **Stop-word removal.** We use the NLTK English stop-word list.
6. **Short-token filtering.** Single letters are dropped, but single digits are kept, since they can be meaningful (sizes, pack counts).
7. **Stemming.** We use the Porter stemmer from NLTK, which reduces words such as "pants" and "pant" to one term and improves recall.
8. **Second stop-word filter after stemming.** Some words that are not stop words stem to one (for example "others" → "other", "willing" → "will", "youre" → "your").

Example.

```
d = docs[0]
print(d["title"], "->", build_terms(d["title"]))
print(d["description"])
print(build_terms(d["description"]))
```

0.0s Python

```
Solid Women Multicolor Track Pants -> ['solid', 'women', 'multicolor', 'track', 'pant']
Yorker trackpants made from 100% rich combed cotton giving it a rich look.Designed for Comfort,Skin friendly fabric,itch-free
['yorker', 'trackpant', 'made', '100', 'rich', 'comb', 'cotton', 'give', 'rich', 'look', 'design', 'comfort', 'skin', 'friendl
```

2.2 Output fields (Task 2)

Future query results must return pid, title, description, brand, category, sub_category, product_details, seller, out_of_stock, selling_price, discount, actual_price, average_rating and url when present. We therefore keep the original, unprocessed values of these fields in each document, next to the processed search terms. Stemmed tokens are used only for matching. The user always sees the original text. Missing fields are stored as None instead of an invented value. The pid is never modified, since it is the key used for evaluation. We also built a dictionary from pid to the output fields for fast lookup when displaying ranked results and we decided to keep the numbers as float.

2.3 Handling category, sub_category, brand, product_details and seller (Task 3)

We considered three options.

Option	Pros	Cons
Merge everything into one text field	Simplest index and scoring	Short, distinctive values (fabric, color, fit, brand) are diluted by long descriptions; no per-field weighting; seller names can match queries by accident
Separate fields in the inverted index	Fields can be weighted differently; a match in a structured attribute is stronger evidence than a passing mention in the description	Larger index; needs a rule to combine field scores
Hybrid: separate fields plus a combined field	Good recall from the combined field and precision from the boosted ones	More complexity and tuning

Decision. We index these fields separately (title, description, brand, category and sub_category together, product_details, seller), all with the same analyzer, and also keep a combined bag of words for baselines.

Justification.

- **product_details** is flattened from a list of key/value pairs into text such as "Fabric Cotton Blend Color Multicolor". It holds the attributes that the evaluation queries ask for (fabric, sleeve, color, fit), so it deserves its own, highly weighted field. We drop identifier-like keys (for example Style Code) because they add noise.
- **category and sub_category** have few distinct values, so most of their terms have low IDF and little discriminative power. We keep them as a low-cost field to help with queries such as "jeans" or "bottomwear".
- **brand** is highly distinctive but rarely appears in the queries, so it is a separate field that only matches when a brand is named.
- **seller** rarely helps retrieval, so it gets a low weight or is only returned in the output.

2.4 Numeric and boolean fields (Task 4)

out_of_stock, selling_price, discount, actual_price and average_rating are **not indexed as text terms**. Indexing "921" as a word would create a term with no semantic meaning. Instead:

- Prices, discount and rating are converted to numbers ("2,999" → 2999.0, "69% off" → 69.0, "3.9" → 3.9), so they can be used for range filters, sorting and small ranking boosts.
- out_of_stock stays a boolean, usable as a filter or a small ranking penalty.
- Missing values stay as None, not 0, so that a missing price is not confused with a free product.

3. Exploratory data analysis

For each analysis we state what we computed and what we conclude.

3.1 Missing values and duplicates

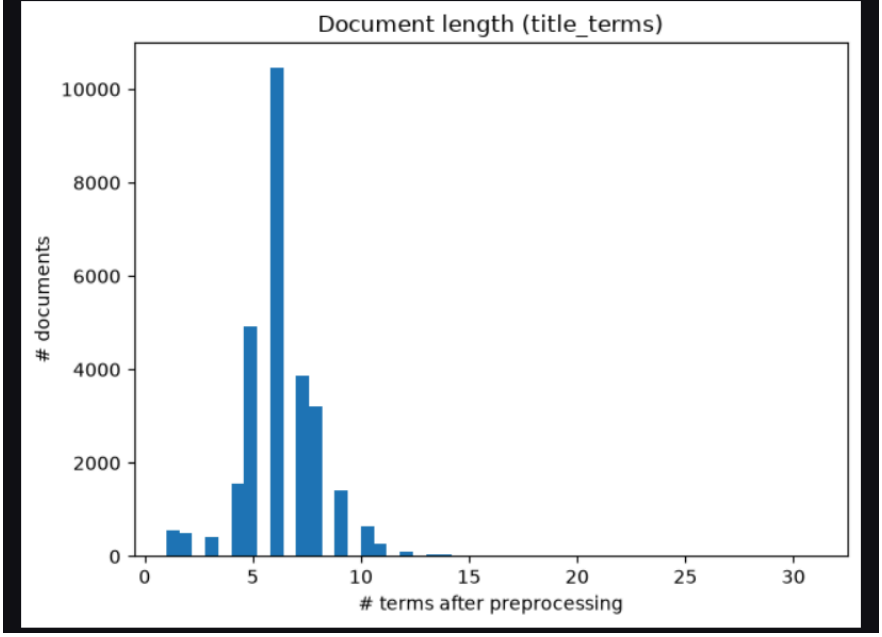
The following table shows the percentage of missing values per field, number of duplicated pids, number of documents with empty processed title. We observed 11,150 empty processed titles meaning that the data simply has no title for those products. That is a property of the dataset, not a bug.

```
title            0.00
description      0.00
brand            0.00
category         0.00
sub_category     0.00
seller           0.00
selling_price    0.01
actual_price     2.77
discount         3.04
average_rating   8.05
dtype: float64
Duplicated pids: 0
Empty term lists: {'title_terms': 0, 'description_terms': 11150, 'details_terms': 50}
```

3.2 Vocabulary and document length

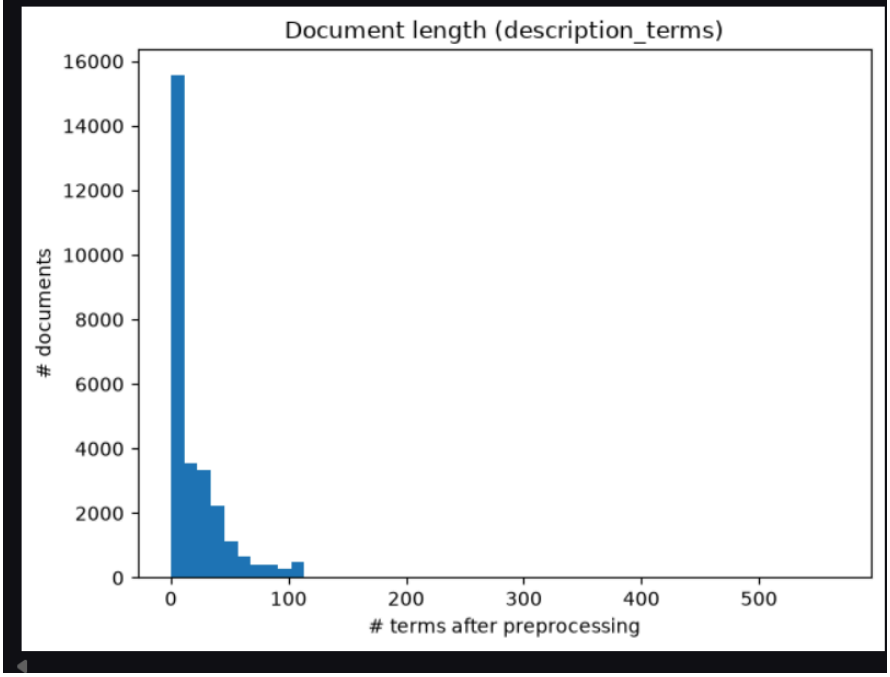
Most titles are short and consistent, centered around roughly 6 terms, with few longer outliers.

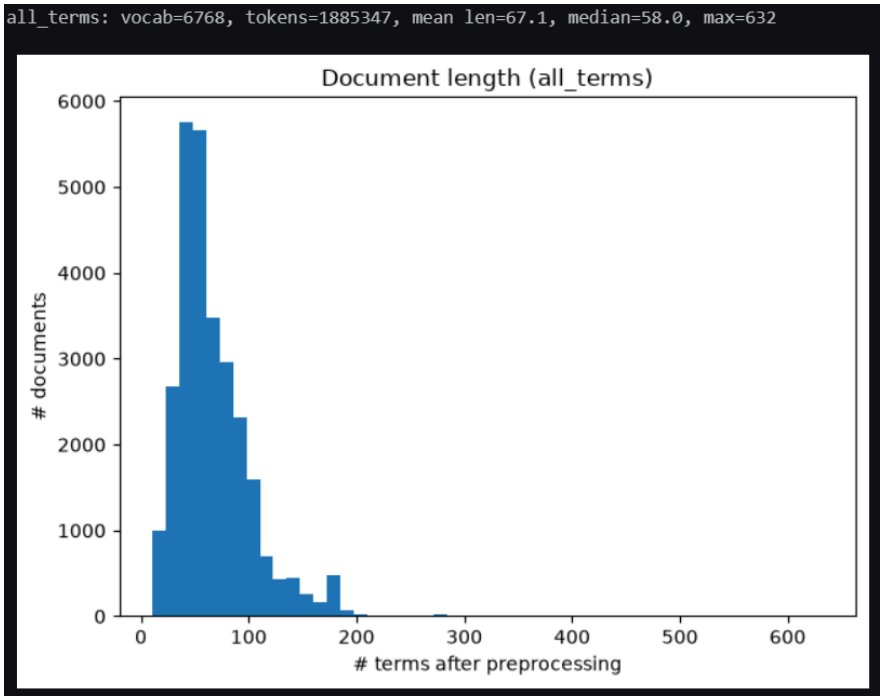
```
title_terms: vocab=731, tokens=175495, mean len=6.2, median=6.0, max=31
```



Very right-skewed. Most descriptions are short, but a small number are much longer.

```
description_terms: vocab=4681, tokens=527535, mean len=18.8, median=9.0, max=567
```

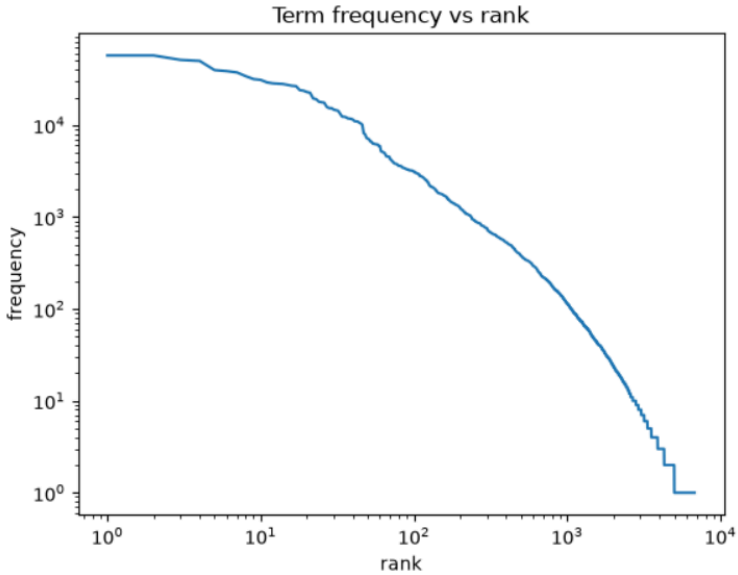




Documents are generally around 58–67 terms, with a long tail of unusually detailed documents.

3.3 Word frequency distribution

A small number of terms occur very frequently, while most terms are rare. The approximately straight declining trend on the loglog scale indicates that term frequency decreases rapidly as rank increases, producing a long tail of infrequent vocabulary. Thanks to Claude, we discovered that this is a Zipf distribution.

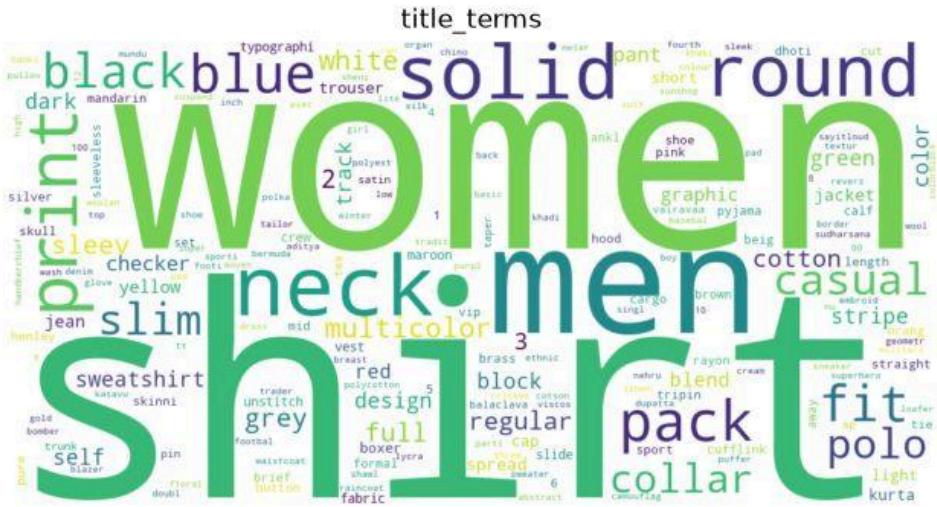


3.4 Sentence length

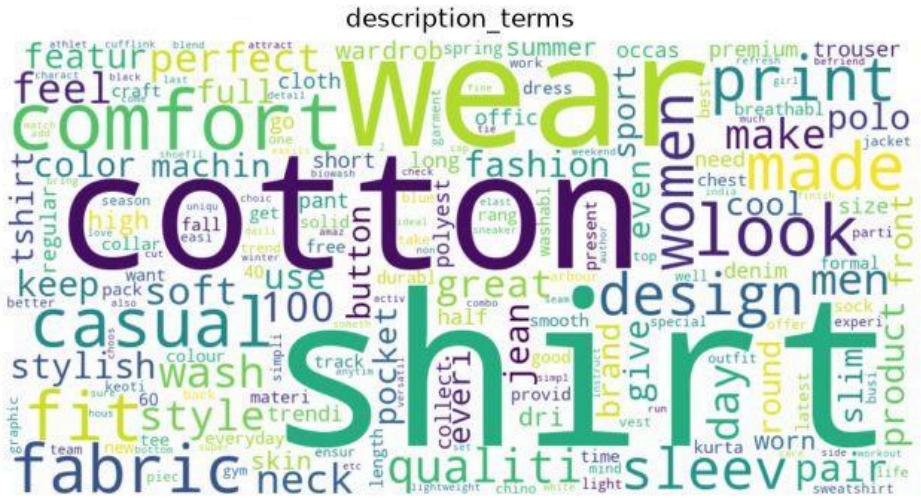
```
Average sentence length (words): 15.85
Average description length (words): 29.8
```

3.5 Word clouds

The two most frequent titles are women and shirt.

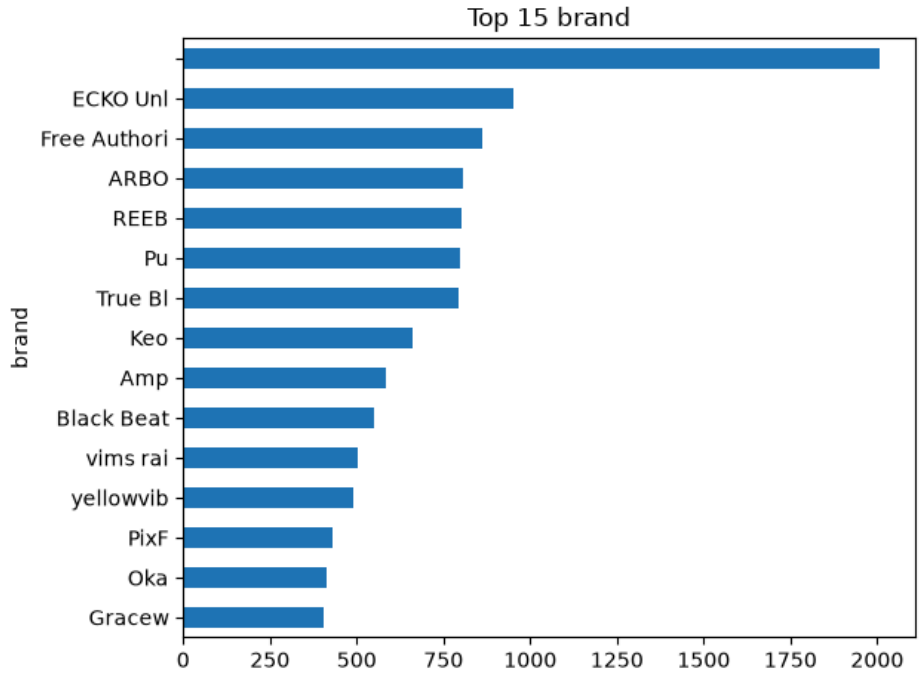


The three most frequent words in descriptions are wear, cotton and shirt.

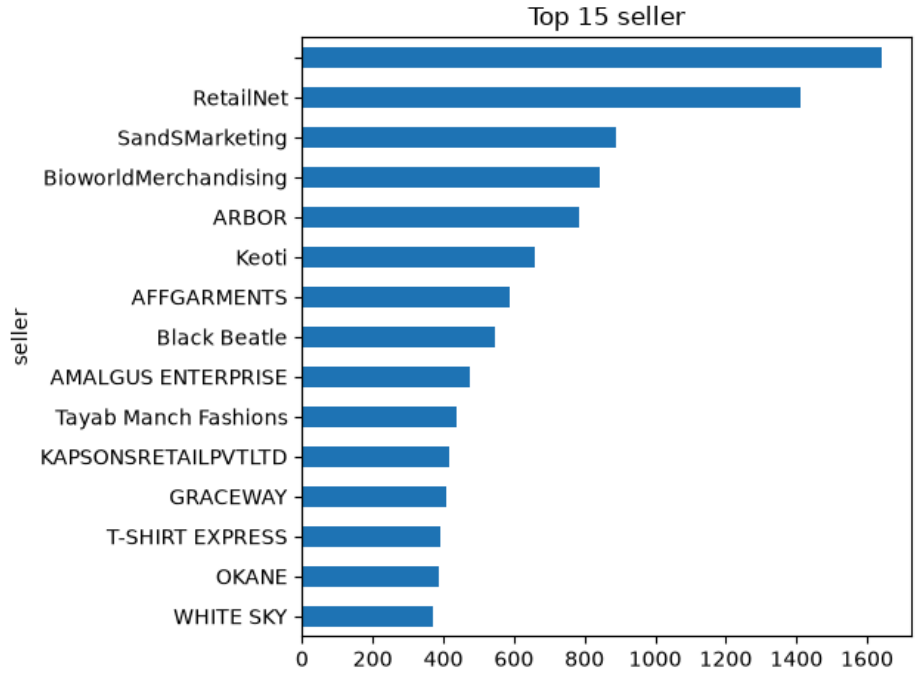


3.6 Brands, sellers and categories

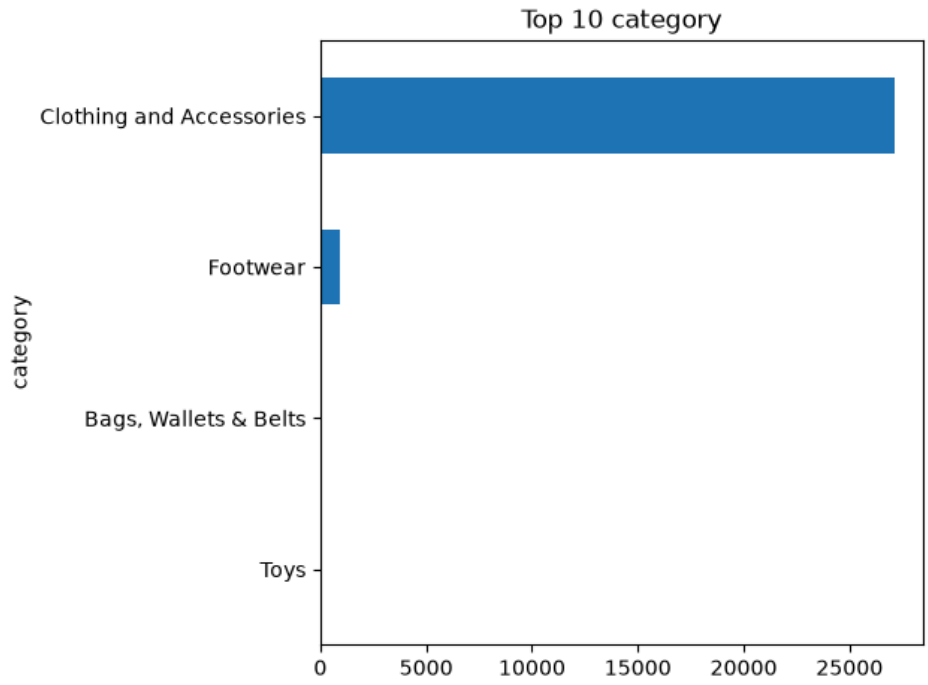
Top brad is ECKO Unl



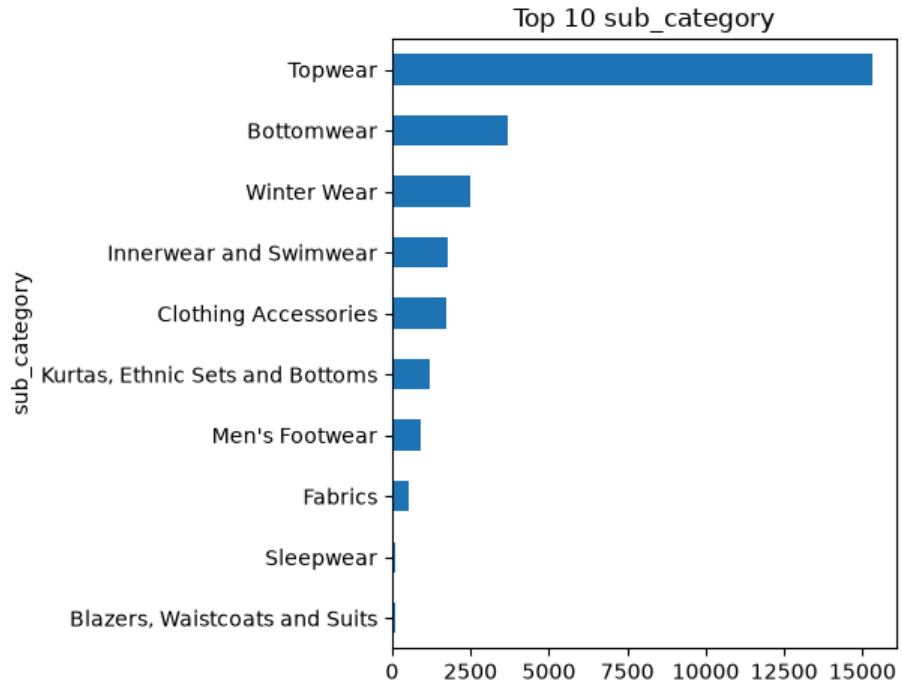
The top seller is RetailNet.



Top 1 category is Clothing and Accessories.

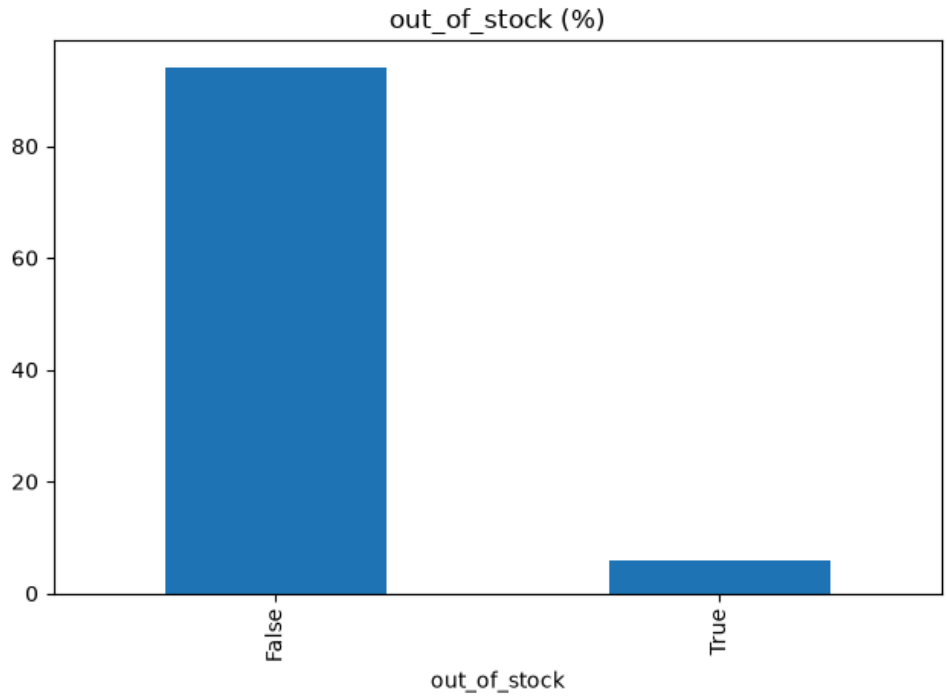


Top sub_category is Topwear.



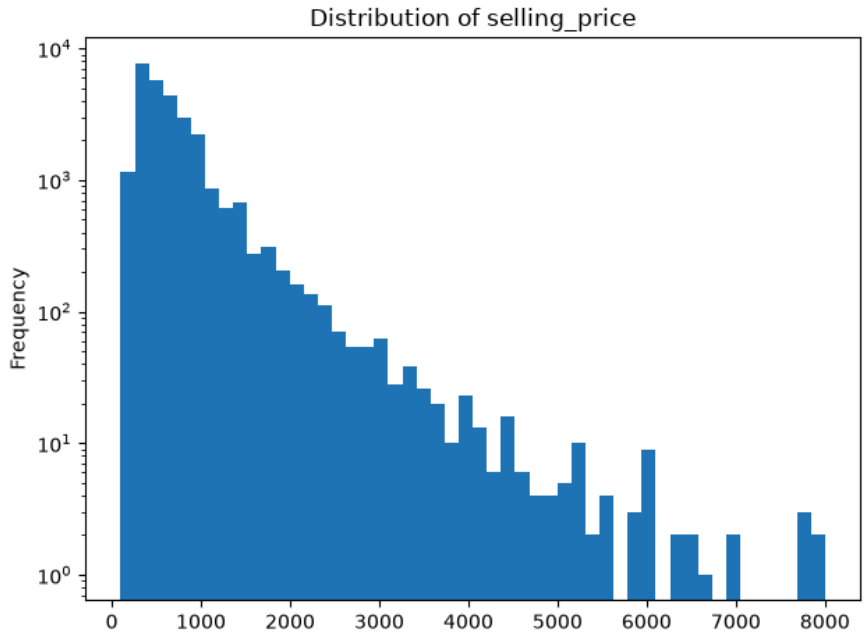
3.7 Stock availability

The following plot shows that 94.15% of products are in stock, while only 5.85% are out of stock. Thus, stock availability is generally high, with a small minority of unavailable products.

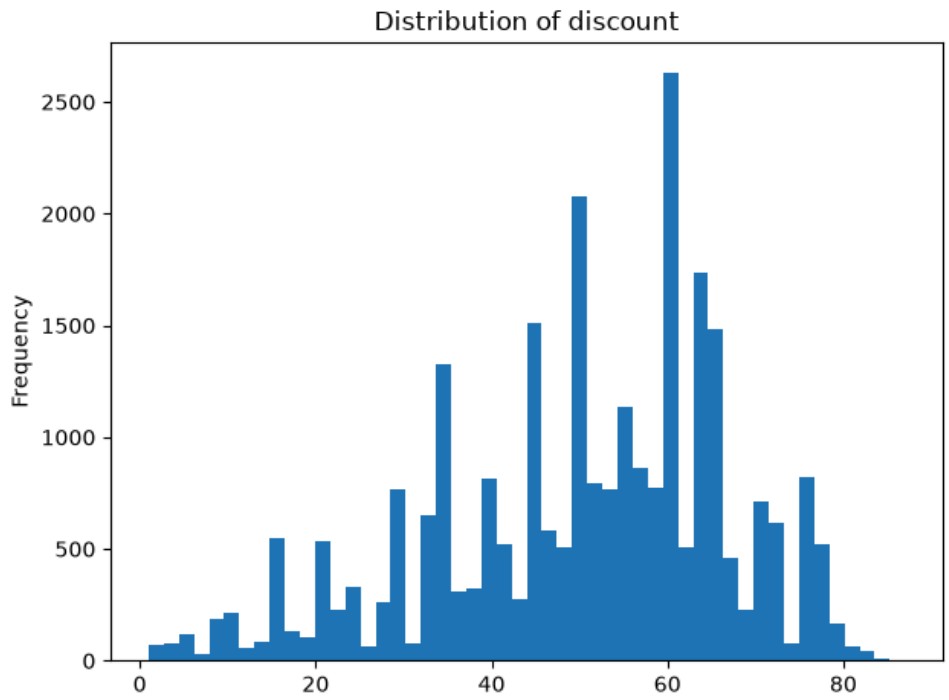


3.8 Prices, discount and rating

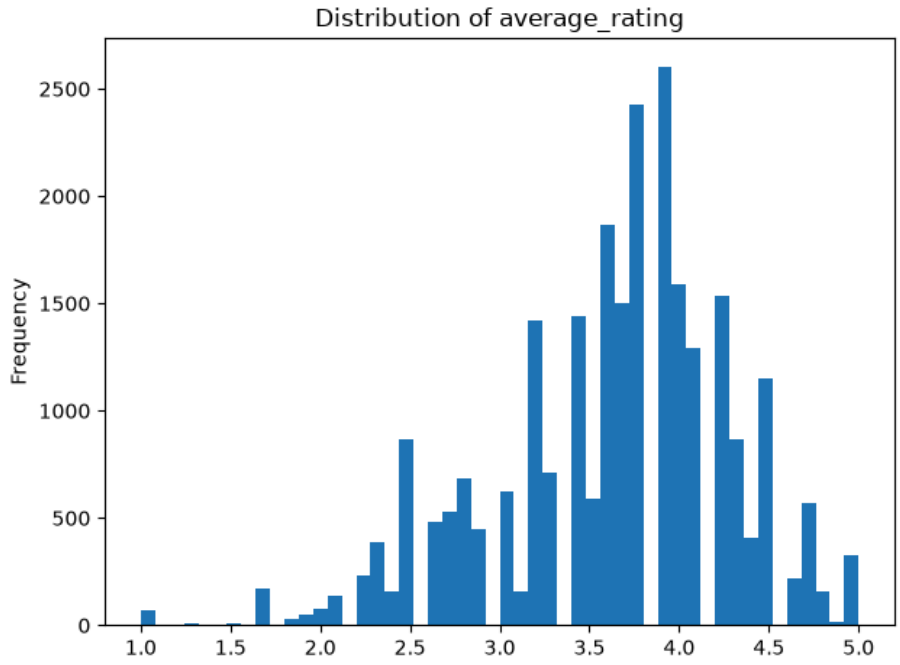
Prices: Selling prices are strongly right-skewed: most products are relatively affordable (median 545), with a few very expensive outliers up to 7,999. Actual prices show the same pattern.



Discounts: Discounts are generally substantial, with a median of about 53%, suggesting frequent promotional pricing.



Ratings: Ratings are centered around 3.8/5, indicating moderate-to-good customer satisfaction, but the distribution includes some unrated products.



3.9 Named entity recognition

```
[(('PERCENT', '100%'), 215), (('PERCENT', '60%'), 85), (('PERCENT', '40%'), 80), (('GPE', 'Cotton'), 69), (('GPE', 'kurta'), 57), (('CARDINAL',
[('ORG', 1041), ('PERSON', 743), ('PERCENT', 401), ('DATE', 387), ('GPE', 335), ('CARDINAL', 251), ('NORP', 140), ('WORK_OF_ART', 138), ('TIME',
```

The NER model most often detected organizations, people, percentages, and dates. However, it frequently misclassified fashion terms such as “Cotton,” “Kurta,” and “Jeans,” so a fashion-specific NER model would be more accurate.

4. Conclusions and implications for the next phases

- **Most informative fields:** Title and description should be the main searchable text fields because they contain product names, characteristics, materials, and uses. Brand, category, and sub_category are useful for exact filtering and metadata-based matching. product_details can provide additional descriptive information.
- **Missing data:** Missing text values should be replaced with empty strings so they do not generate indexing errors. Missing brand, seller, or category values should be treated as “Unknown” or excluded from metadata filters. Missing ratings should not be interpreted as zero.
- **Numeric fields:** selling_price, actual_price, and discount can be used for range filters and optionally as ranking boosts. Average_rating can improve ranking, but only when present. out_of_stock should primarily be used as a filter or penalty so unavailable products appear lower in the results.

5. Reproducibility

The README in the repository root gives step-by-step instruction.

6. AI usage statement

We have used Claude to brainstorm pre-processing strategies, to draft the initial pre-processing, EDA code and to know which libraries to use in every case. It has also helped us debug path and validation issues. Moreover, to create the README.

We reviewed, tested and modified the code on the real dataset and we wrote the justifications in this report ourselves.